

Intergiciels

Examen

Daniel Hagimont

Durée: 1 heures 45 minutes, documents autorisés

*Lire l'ensemble des énoncés avant de commencer à répondre. La **clarté**, la **précision** et la **concision** des réponses, ainsi que leur **présentation matérielle**, seront des éléments importants d'appréciation. Donnez essentiellement les programmes Java demandés. Dans vos programmes, vous n'avez pas à programmer les imports et le traitement des exceptions.*

PROBLÈME I (sockets)

Lorsque des abonnés à Internet sont connectés par la technologie ADSL (Asymétric Digital Subscriber Line), le débit montant (upload) est nettement inférieur au débit descendant (download). Ceci implique que si UsagerClient télécharge un gros document donné par UsagerServer, le transfert est limité par le débit montant (upload) de UsagerServer. Pour diminuer le temps de transfert, il est possible pour UsagerClient de lancer le téléchargement de différents fragments du même document à partir de plusieurs usagers possédant une copie du document. Cette technique est utilisée par de nombreuses plate-formes de téléchargement dites P2P.

Nous voulons implanter le schéma de principe de cette technique. Vous devez programmer avec des sockets en Java deux programmes exécutés respectivement sur le client et sur les machines serveurs :

- UsagerClient télécharge des fragments d'un document depuis plusieurs serveurs
- UsagerServer_i fournit les fragments du document

Question 1

Quelle propriété essentielle doit remplir UsagerClient pour que cette technique puisse bénéficier de l'existence de plusieurs serveurs pour accélérer le téléchargement ?

Cette propriété est elle nécessaire pour les processus UsagerServer_i ?

Question 2

On suppose que les programmes UsagerClient et UsagerServer_i ont les variables suivantes définies :

```
final static String hosts[] = {"host1", "host2", "host3"};  
final static int ports[] = {8081,8082,8083};  
final static int nb = 3;  
static String document[] = new String[nb];
```

Les 3 premières variables donnent les adresses des serveurs. Un serveur démarre en prenant en paramètre son numéro de serveur (0, 1 ou 2, l'indice dans les tables *hosts* et *ports*). On ne gère qu'un seul document qui est ici un tableau de *String* (*document*) et on considère qu'un fragment est une *String* de ce tableau. On suppose que le tableau *document* est initialisé pour les serveurs (vous n'avez pas à le faire) et il doit être rempli par le téléchargement pour le client.

On simplifie donc ce schéma avec 1 document composé de 3 *String*, téléchargées depuis 3 serveurs.

Pour les requêtes, il est recommandé d'utiliser la sérialisation d'objets Java. Une requête peut simplement envoyer un numéro de fragment sur une connexion TCP et recevoir la *String* correspondant au fragment.

Donnez une implantation des programmes UsagerClient et UsagerServer_i en suivant ce schéma.

PROBLÈME II (RMI)

Vous avez à réaliser un service d'exécution de commande à distance (comme rsh) en utilisant Java RMI.

Ce service est composé de 2 programmes :

- un programme `Daemon.java` qui doit être lancé sur toutes les machines vers lesquels on veut pouvoir lancer des commandes à distance. Ce programme enregistre un objet RMI sur son `RMIRegistry` local.
- Un programme `RE.java` qui permet l'exécution d'une commande à distance. Ce programme interagit avec le programme `Daemon` du site sur lequel il faut exécuter la commande.

Le programme `RE` s'utilise de la façon suivante : `java RE <host> <command>`

Le programme `RE` reçoit dans `args[0]` et `args[1]` les 2 chaînes de caractères `host` et `command`.

Pour exécuter une commande localement sur les machines distantes, on utilise : `localExec(String cmd);`

Question 3

Donnez une implantation en Java des programmes `Daemon` et `RE`.

Question 4

On veut maintenant gérer l'affichage console de la commande exécutée à distance. On veut que les affichages de la commande soient effectués sur la console du client qui a lancé la commande.

On exécute maintenant une commande localement avec : `localExec(String command, Console console);`

`Console` est une interface Java qui fournit notamment la méthode `println(String s);`

La commande exécutée réalise ses affichages en appelant cette méthode sur l'objet `Console` passé en paramètre de `localExec()`.

Modifiez votre implantation précédente pour permettre l'affichage sur le poste client de la commande exécutée à distance.

```

1 // Problème 1
2
3 // Question 1 :
4 // Il faut que tout les clients soient parallélisés :
5 //     on télécharge chacun des morceaux de code en même temps
6 // Pas de parallélisation pour un serveur :
7 //     il ne pourra dans tous les cas fournir qu'une seule ressource
8 //     puisqu'il expose un socket sur un seul port
9
10 public class UsagerServer {
11     final static String hosts[] = {"host1", "host2", "host3"};
12     final static int nb = 3;
13     static String documents[] = new String[nb];
14
15     public static void main(String args[]) {
16         ServerSocket ss = new ServerSocket(8080);
17
18         while (true) {
19             Socket s = ss.accept();
20             OutputStream os = s.getOutputStream();
21             InputStream is = s.getInputStream();
22             ObjectOutputStream oos = new ObjectOutputStream(os);
23             ObjectInputStream ois = new ObjectInputStream(is);
24
25             int num = (int)ois.readObject();
26             oos.writeObject(documents[num]);
27         }
28     }
29 }
30
31 public class UsagerClient extends Thread {
32     final static String hosts[] = {"host1", "host2", "host3"};
33     final static int ports[] = {8081, 8082, 8083};
34     final static int nb = 3;
35     static String documents[] = new String[nb];
36
37     int target;
38
39     public UsagerClient(int i) {
40         target = i;
41     }
42
43     public void run() {
44         Socket s = new Socket(hosts[target], ports[target]);
45         OutputStream os = s.getOutputStream();
46         InputStream is = s.getInputStream();
47         ObjectOutputStream oos = new ObjectOutputStream(os);
48         ObjectInputStream ois = new ObjectInputStream(is);
49
50         oos.writeObject(target);
51         documents[target] = (String)ois.readObject();
52         s.close();
53     }
54
55     public static void main(String args[]) {
56         Thread th[] = new Thread[nb];

```

```

57         for (int i=0 ; i<nb ; i++) {
58             th[i] = new UsagerClient(i);
59             th[i].start();
60         }
61
62         for (int i=0 ; i<nb ; i++) {
63             th[i].join();
64         }
65     }
66 }
67
68
69
70
71 // Problème 2
72
73 // Ajouts pour la console en commentaire
74 // Visiblement on appelle ça une "inversion de contrôle"
75
76 public interface Daemon extends Remote {
77     public void exec(String cmd /* , Console console */) throws RemoteException;
78 }
79
80 public class DaemonImpl extends UnicastRemoteObject implements Daemon {
81     public DaemonImpl() {}
82
83     public void exec(String cmd /* , Console console */) throws RemoteException {
84         localExec(cmd /*, console */);
85     }
86
87     public static void main(String args[]) {
88         LocalRegistry.createRegistry(2000);
89         Naming.bind("//localhost:2000/daemon", new DaemonImpl());
90     }
91 }
92
93 public class RE {
94     public static void main(String args[]) {
95         // On met bien Daemon et pas DaemonImpl sinon on se prend -10pts
96         Daemon d = (Daemon)Naming.lookup("//"+args[0]+":2000/daemon");
97         d.exec(args[1] /*, new ConsoleImpl() */);
98     }
99 }
100
101 public interface Console extends Remote {
102     public void println(String s) throws RemoteException;
103 }
104
105 public class ConsoleImpl extends UnicastRemoteObject implements Console {
106     public Console() throws RemoteException {}
107
108     public void println(String s) throws RemoteException {
109         System.out.println(s);
110     }
111 }
112

```